

Assignment No. 01

Course Title: CS-426L Parallel and Distributed Computing Lab

TITLE OF THE ASSIGNMENT

" Implementation of Core OOP Architectures, Array-Based Stacks & Queues, and Thread-Based Concurrency in Java "

SUBMITTED BY:

Name: Usama Arshad

Student ID: 247141

Semester: 5th

Section: BS-CYS-F24-A

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: October 1, 2026

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF Computer Science

Question 1: GitHub Webhook Security Utils (String Reversal, Digit Sum, Factorial)

1. Approach / Explanation

I developed two separate files: CommitSecurityUtils.java and GitHubWebhookApp.java. This allowed me to isolate reusable security logic from the main execution driver. Within CommitSecurityUtils, I implemented the reverseHash method using a descending for loop that constructs a String by appending each character. I also created calculatePayloadSum, which checks if each char falls within the range of **0** to **9** and accumulates its integer value. Additionally, I included generateSecurityToken, which utilizes an ascending for loop to calculate the factorial of a long variable from 1 to the digit sum. In the main method, I preserved the sample hash and payload while invoking each method. This setup allowed me to verify in the terminal that the reversed hash, the extracted digit sum, and the factorial token all worked as intended, confirming that everything was functioning properly.

2. Code

File: *CommitSecurityUtils.java*

```java
public class CommitSecurityUtils {

    // Method to reverse a commit hash string using a basic loop
    public String reverseHash(String commitHash) {
        String reversed = "";
        for (int i = commitHash.length() - 1; i >= 0; i--) {
            reversed += commitHash.charAt(i);
        }
        return reversed;
    }

    // Method to extract digits and sum them up
    public int calculatePayloadSum(String payload) {
        int sum = 0;
        for (int i = 0; i < payload.length(); i++) {
            char ch = payload.charAt(i);
            if (ch >= '0' && ch <= '9') {
                sum += (ch - '0'); // Convert char digit to integer
            }
        }
        return sum;
    }

    // Method to compute factorial of the sum
    public long generateSecurityToken(int number) {
        long factorial = 1;
        for (int i = 1; i <= number; i++) {
            factorial *= i;
        }
        return factorial;
    }
}
```

File: *GitHubWebhookApp.java*

```java
public class GitHubWebhookApp {
    public static void main(String[] args) {
        CommitSecurityUtils utils = new CommitSecurityUtils();

        String sampleHash = "a1b2c3d4";
        String samplePayload = "commit_id_9021";

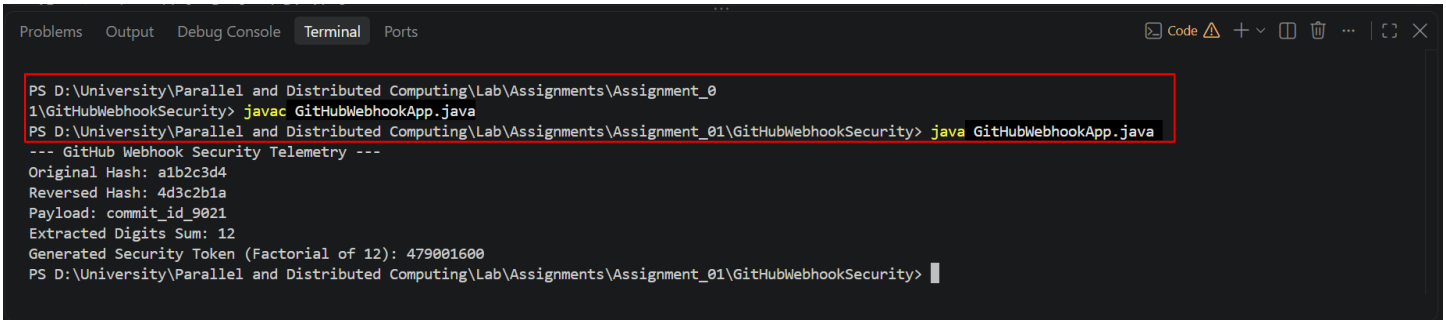
        System.out.println("--- GitHub Webhook Security Telemetry ---");

        // 1. Reverse Hash
        String reversed = utils.reverseHash(sampleHash);
        System.out.println("Original Hash: " + sampleHash);
        System.out.println("Reversed Hash: " + reversed);

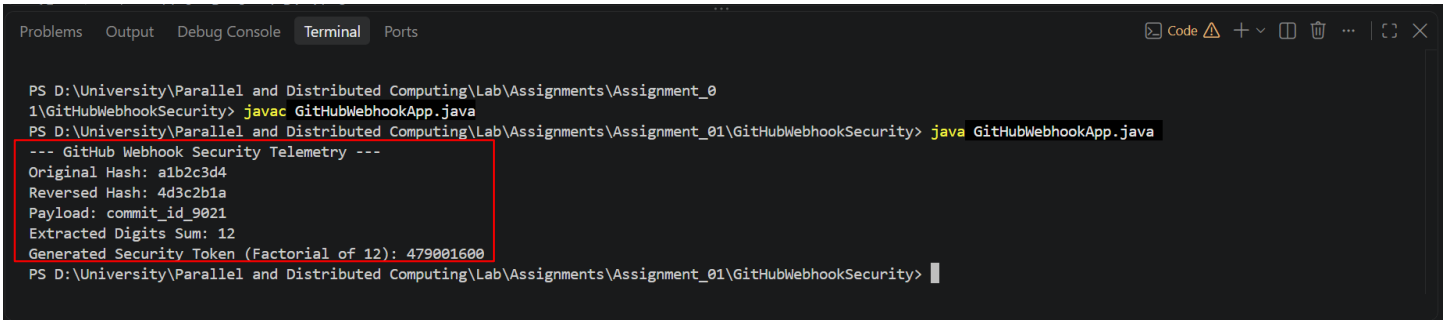
        // 2. Payload Digit Sum
        int digitSum = utils.calculatePayloadSum(samplePayload);
        System.out.println("Payload: " + samplePayload);
        System.out.println("Extracted Digits Sum: " + digitSum);

        // 3. Security Token (Factorial)
        long token = utils.generateSecurityToken(digitSum);
        System.out.println("Generated Security Token (Factorial of " + digitSum + "): " + token);
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



Question 2: LinkedIn Notification Dispatch (Inheritance & Method Overriding)

1. Approach / Explanation

I designed ActivityNotification as the base class, incorporating protected fields for senderName and timestamp, along with a constructor and a dispatch method. I then created subclasses, ConnectionRequest and JobAlert, which extend the base class by adding their own private fields. Each subclass makes use of the super(senderName, timestamp) call in their constructors and overrides the dispatch method to print specialized notification messages. In LinkedInActivityApp, I defined ActivityNotification references that point to both ConnectionRequest and JobAlert instances, showcasing runtime polymorphism. Upon running the program, the terminal displayed the unique dispatch outputs for each object, confirming that method overriding and dynamic binding were working effectively.

2. Code

File: *ActivityNotification.java*

```
public class ActivityNotification {
    protected String senderName;
    protected String timestamp;

    public ActivityNotification(String senderName, String timestamp) {
        this.senderName = senderName;
        this.timestamp = timestamp;
    }

    public void dispatch() {
        System.out.println("[" + timestamp + "] Generic Notification from " + senderName);
    }
}
```

File: *ConnectionRequest.java*

```
public class ConnectionRequest extends ActivityNotification {
    private String connectionNote;

    public ConnectionRequest(String senderName, String timestamp, String connectionNote) {
        super(senderName, timestamp);
        this.connectionNote = connectionNote;
    }

    @Override
    public void dispatch() {
        System.out.println("[" + timestamp + "] Connection Request from " + senderName
                + " | Note: \"" + connectionNote + "\"");
    }
}
```

File: *JobAlert.java*

```java
public class JobAlert extends ActivityNotification {
    private String jobTitle;
    private String companyName;

    public JobAlert(String senderName, String timestamp, String jobTitle, String companyName) {
        super(senderName, timestamp);
        this.jobTitle = jobTitle;
        this.companyName = companyName;
    }

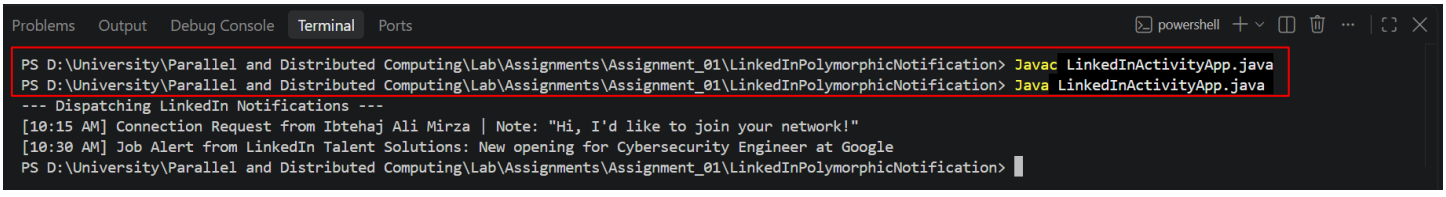
    @Override
    public void dispatch() {
        System.out.println("[" + timestamp + "] Job Alert from " + senderName
                + ": New opening for " + jobTitle + " at " + companyName);
    }
}
```

File: *LinkedInActivityApp.java*

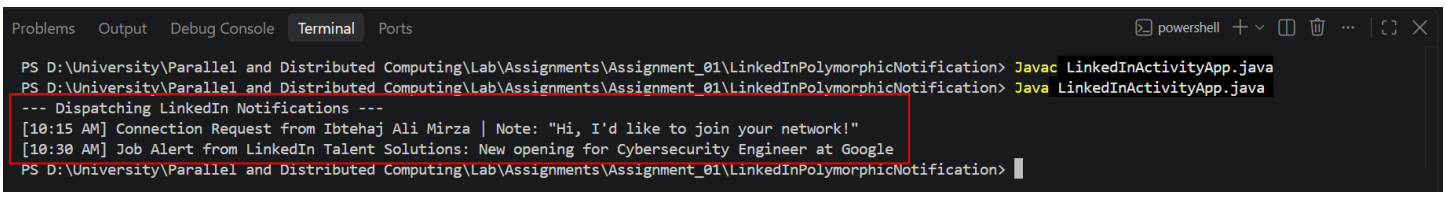
```java
public class LinkedInActivityApp {
    public static void main(String[] args) {
        // Polymorphism: Superclass reference variables pointing to Subclass objects
        ActivityNotification notif1 = new ConnectionRequest("Ibtehaj Ali Mirza", "10:15 AM",
                "Hi, I'd like to join your network!");
        ActivityNotification notif2 = new JobAlert("LinkedIn Talent Solutions", "10:30 AM",
                "Cybersecurity Engineer", "Google");

        System.out.println("--- Dispatching LinkedIn Notifications ---");
        // Runtime method overriding executes appropriate subclass logic
        notif1.dispatch();
        notif2.dispatch();
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



Question 3: Browser Tab History (Array-Based Stack)

1. Approach / Explanation

I created URLStack to implement a fixed-capacity stack utilizing a String array, an index called top, and a capacity variable. This class includes methods for push, pop, peek, as well as isFull and isEmpty. Following this, I developed TabHistoryManager, which maintains backStack, forwardStack, and currentURL. Here, visitURL pushes the old URL onto the back stack while clearing the forward stack, goBack moves the current URL to the forward stack and pops from the back stack, and goForward reverses this process. WebBrowserApp simulates visiting three different URLs, then going back twice and forward once. The output in the terminal aligned perfectly with the expected navigation sequence, confirming that the stack was functioning with Last-In-First-Out (LIFO) behavior and effectively managing history.

2. Code

File: *URLStack.java*

```
public class URLStack {
    private String[] stackArray;
    private int top;
    private int capacity;

    public URLStack(int capacity) {
        this.capacity = capacity;
        this.stackArray = new String[capacity];
        this.top = -1;
    }

    public boolean isFull() {
        return top == capacity - 1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public void push(String url) {
        if (isFull()) {
            System.out.println("Stack Overflow: Cannot push " + url);
            return;
        }
        top++;
        stackArray[top] = url;
    }

    public String pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow: No URLs to pop.");
            return null;
        }
        String popped = stackArray[top];
        top--;
```

```
        return popped;
    }

    public String peek() {
        if (isEmpty()) {
            return null;
        }
        return stackArray[top];
    }
}
```

File: *TabHistoryManager.java*

```
public class TabHistoryManager {
    private URLStack backStack;
    private URLStack forwardStack;
    private String currentURL;

    public TabHistoryManager(int maxCapacity) {
        backStack = new URLStack(maxCapacity);
        forwardStack = new URLStack(maxCapacity);
        currentURL = "about:blank";
    }

    public void visitURL(String url) {
        if (currentURL != null && !currentURL.equals("about:blank")) {
            backStack.push(currentURL);
        }
        currentURL = url;
        // Clear forward history on new navigation
        while (!forwardStack.isEmpty()) {
            forwardStack.pop();
        }
        System.out.println("Visited: " + currentURL);
    }

    public void goBack() {
        if (backStack.isEmpty()) {
            System.out.println("Cannot go back: Back history is empty.");
            return;
        }
        forwardStack.push(currentURL);
        currentURL = backStack.pop();
        System.out.println("Went Back to: " + currentURL);
    }

    public void goForward() {
        if (forwardStack.isEmpty()) {
            System.out.println("Cannot go forward: Forward history is empty.");
            return;
        }
```

```
        backStack.push(currentURL);
        currentURL = forwardStack.pop();
        System.out.println("Went Forward to: " + currentURL);
    }

    public String getCurrentURL() {
        return currentURL;
    }
}
```

File: *WebBrowserApp.java*

```
public class WebBrowserApp {
    public static void main(String[] args) {
        TabHistoryManager browserTab = new TabHistoryManager(10);

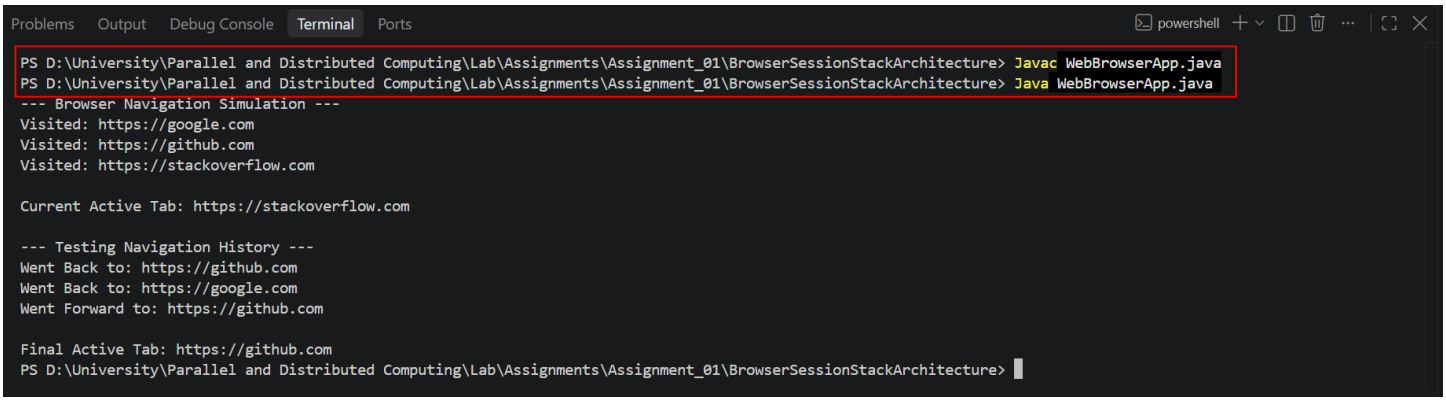
        System.out.println("--- Browser Navigation Simulation ---");
        browserTab.visitURL("https://google.com");
        browserTab.visitURL("https://github.com");
        browserTab.visitURL("https://stackoverflow.com");

        System.out.println("\nCurrent Active Tab: " + browserTab.getCurrentURL());

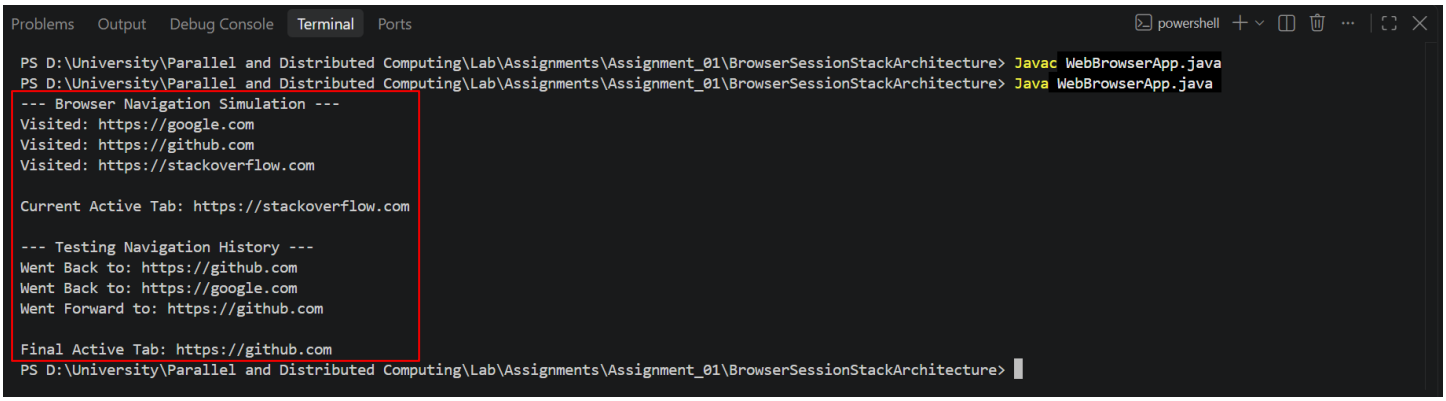
        System.out.println("\n--- Testing Navigation History ---");
        browserTab.goBack(); // Returns to github.com
        browserTab.goBack(); // Returns to google.com
        browserTab.goForward(); // Returns to github.com

        System.out.println("\nFinal Active Tab: " + browserTab.getCurrentURL());
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



Question 4: CI Job Queue (Array-Based Queue, FIFO)

1. Approach / Explanation

I designed JobQueue using a String array alongside variables for front, rear, size, and capacity. The enqueue method adds an item at the rear, while dequeue removes from the front, shifting the remaining elements to the left. Subsequently, I developed CIPipelineDispatcher, which wraps the queue and implements methods like submitBuildJob, processNextJob, and showPipelineStatus. The GitHubActionsApp simulates continuous integration (CI) jobs. In the main method, I submitted four jobs, displayed the queue's status, processed two jobs in a First-In-First-Out (FIFO) manner, and showcased the updated queue. The terminal output indicated that the jobs were processed in the exact order they were submitted, confirming that the queue maintained its intended FIFO behavior.

2. Code

File: *JobQueue.java*

```
public class JobQueue {
    private String[] queueArray;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public JobQueue(int capacity) {
        this.capacity = capacity;
        this.queueArray = new String[capacity];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    public boolean isFull() {
        return size == capacity;
    }

    public boolean isEmpty() {
        return size == 0;
    }
```

```
    // Add pending jobs at the rear
    public void enqueue(String jobId) {
        if (isFull()) {
            System.out.println("Queue Overflow: Job queue is full.");
            return;
        }
        rear++;
        queueArray[rear] = jobId;
        size++;
    }

    // Remove job from front and shift elements left
    public String dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Underflow: No jobs to process.");
            return null;
        }
        String removedJob = queueArray[front];

        // Shift elements to maintain standard 0-indexed queue
        for (int i = 0; i < rear; i++) {
            queueArray[i] = queueArray[i + 1];
        }
        rear--;
        size--;
        return removedJob;
    }

    public String peek() {
        if (isEmpty()) {
            return null;
        }
        return queueArray[front];
    }

    public void displayQueue() {
        if (isEmpty()) {
            System.out.println("Job Queue is empty.");
            return;
        }
        System.out.print("Pending Jobs [ ");
        for (int i = 0; i <= rear; i++) {
            System.out.print(queueArray[i] + (i < rear ? " -> " : ""));
        }
        System.out.println(" ]");
    }
}
```

File: *CIPipelineDispatcher.java*

```
public class CIPipelineDispatcher {
    private JobQueue jobQueue;

    public CIPipelineDispatcher(int maxCapacity) {
        this.jobQueue = new JobQueue(maxCapacity);
    }

    public void submitBuildJob(String jobName) {
        System.out.println("Submitting Build Job: " + jobName);
        jobQueue.enqueue(jobName);
    }

    public void processNextJob() {
        String job = jobQueue.dequeue();
        if (job != null) {
            System.out.println("Processing & Executing: " + job + " -> Build Artifacts
Generated.");
        }
    }

    public void showPipelineStatus() {
        jobQueue.displayQueue();
    }
}
```

File: *GitHubActionsApp.java*

```
public class GitHubActionsApp {
    public static void main(String[] args) {
        CIPipelineDispatcher dispatcher = new CIPipelineDispatcher(5);

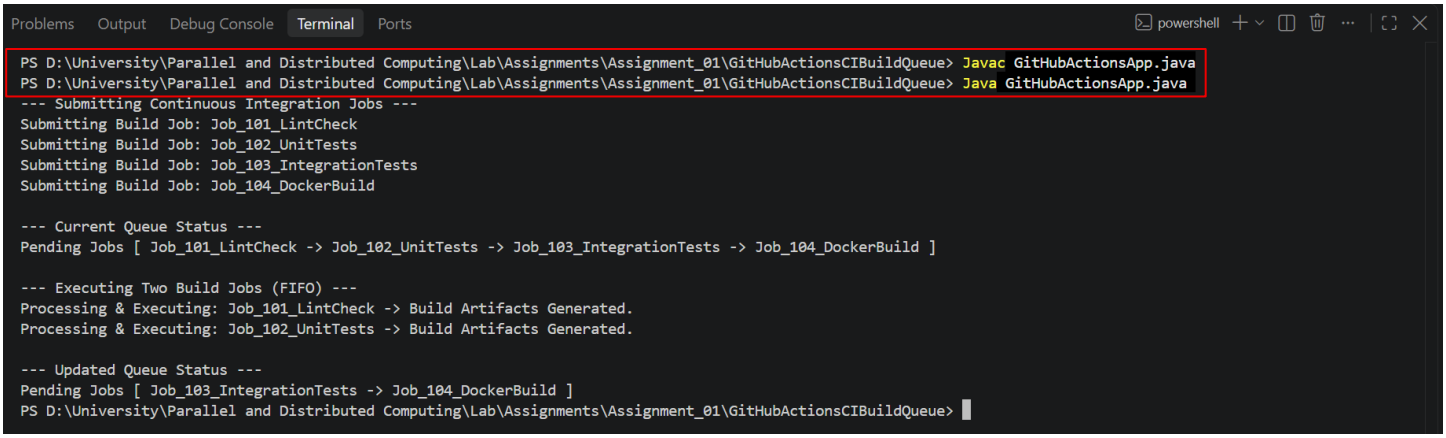
        System.out.println("--- Submitting Continuous Integration Jobs ---");
        dispatcher.submitBuildJob("Job_101_LintCheck");
        dispatcher.submitBuildJob("Job_102_UnitTests");
        dispatcher.submitBuildJob("Job_103_IntegrationTests");
        dispatcher.submitBuildJob("Job_104_DockerBuild");

        System.out.println("\n--- Current Queue Status ---");
        dispatcher.showPipelineStatus();

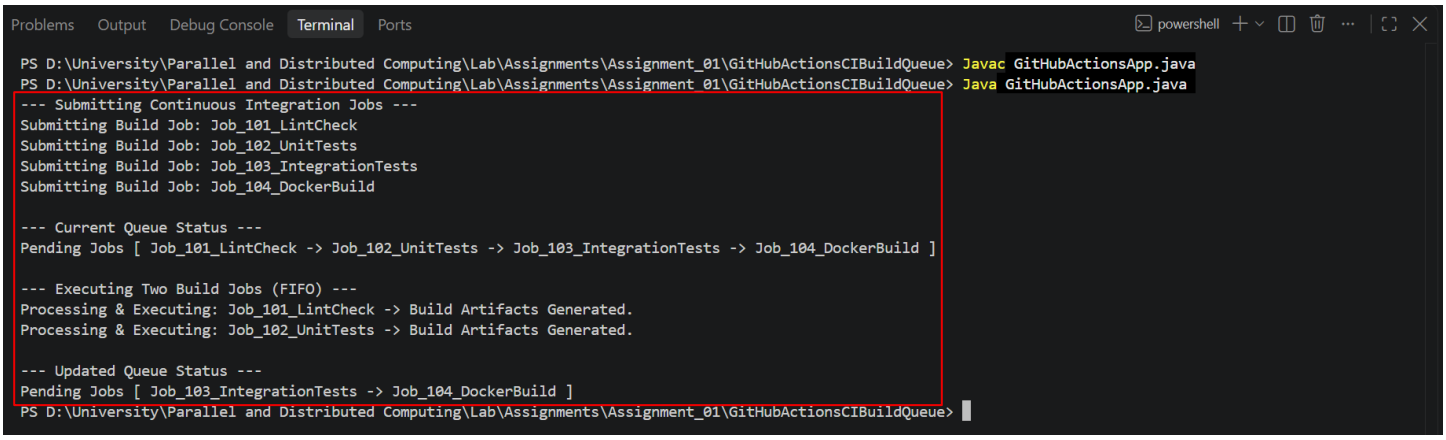
        System.out.println("\n--- Executing Two Build Jobs (FIFO) ---");
        dispatcher.processNextJob();
        dispatcher.processNextJob();

        System.out.println("\n--- Updated Queue Status ---");
        dispatcher.showPipelineStatus();
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



Question 5: Multithreaded Log Processing (Runnable & Thread)

1. Approach / Explanation

I implemented the LogProcessorTask class, which implements Runnable, and the TelemetryThread class, which extends Thread. Each class features a run method that loops five times, printing the current thread's name and progress, while incorporating a sleep interval of 1000ms or 800ms within a try-catch block for InterruptedException. In AsyncWebEngineApp, I displayed the name of the main thread and created both a Telemetry-Worker thread and a Log-Worker thread using the Runnable constructor. When both were started concurrently, I printed their isAlive status. Upon execution, the terminal intermixed the messages from the two threads, confirming that they were running concurrently, and displayed true for the alive status, showcasing proper thread management.

2. Code

File: *LogProcessorTask.java*

```
public class LogProcessorTask implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("[" + Thread.currentThread().getName() + "] Processing log chunk "
+ i + "/5");
            try {
                Thread.sleep(1000); // 1-second pause
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + " was interrupted.");
            }
        }
    }
}
```

File: *TelemetryThread.java*

```
public class TelemetryThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("[" + Thread.currentThread().getName() + "] Telemetry Health Check
" + i
                    + "/5: All services nominal");
            try {
                Thread.sleep(800); // 800ms pause
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + " was interrupted.");
            }
        }
    }
}
```

File: *AsyncWebEngineApp.java*

```java
public class AsyncWebEngineApp {
    public static void main(String[] args) {
        // 1. Output default main thread name assigned by the JVM
        System.out.println("Main Thread Name: " + Thread.currentThread().getName());

        // 2. Create threads using direct inheritance and Runnable interface
        TelemetryThread telemetryWorker = new TelemetryThread();
        telemetryWorker.setName("Telemetry-Worker");

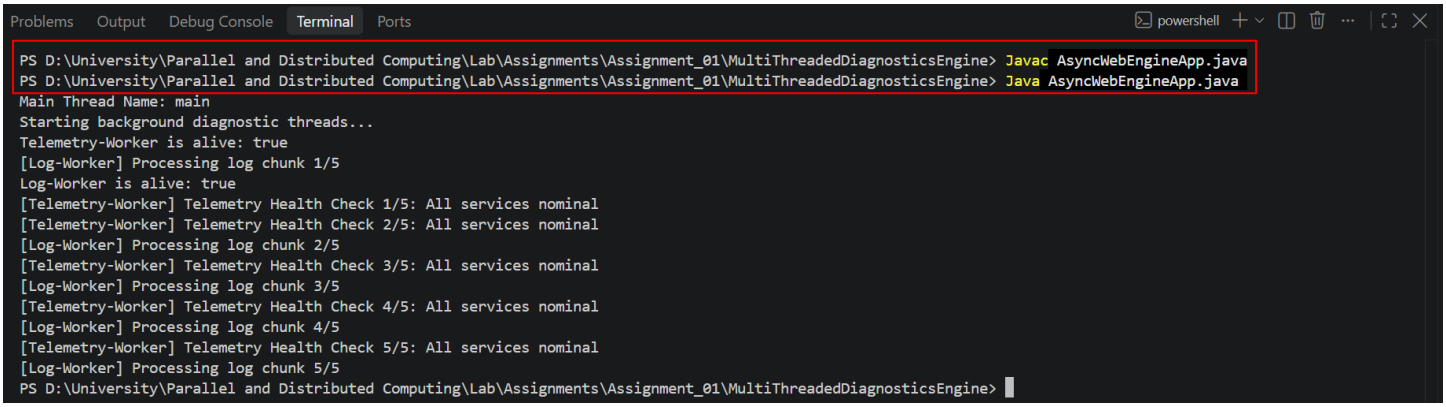
        LogProcessorTask logTask = new LogProcessorTask();
        Thread logWorker = new Thread(logTask, "Log-Worker");

        System.out.println("Starting background diagnostic threads...");

        // 3. Start threads concurrently
        telemetryWorker.start();
        logWorker.start();

        // 4. Log active execution states
        System.out.println(telemetryWorker.getName() + " is alive: " + telemetryWorker.isAlive());
        System.out.println(logWorker.getName() + " is alive: " + logWorker.isAlive());
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot

